

PI IV - DESENVOLVIMENTO PARA DISPOSITIVOS MÓVEIS

Prof. Me. Wilson Lourenço

React Native: Estados com useState

- Gerenciamento de estado em componentes funcionais, com exemplos práticos no Expo Go



O que é useState?



useState: O Coração do Estado em React



Hook Fundamental

- Permite adicionar estado a componentes funcionais de forma simples e intuitiva



Persistência de Dados

- Mantém valores entre renderizações sem perder informações



Atualização Automática

- Atualiza a interface do usuário automaticamente quando o estado muda

Como Declarar um Estado com useState

Sintaxe Básica

- A declaração do useState segue um padrão de desestruturação de array, onde o primeiro elemento é a variável de estado e o segundo é a função atualizadora.

Exemplo Prático

- No exemplo abaixo, criamos um contador iniciando em zero. A variável `count` armazena o valor atual, enquanto `setCount` é a função para atualizá-lo.

```
const [count, setCount] = useState(0)
```

Anatomia da Declaração useState

01

Importação do Hook

- Importar useState da biblioteca React

02

Desestruturação do Array

- Declarar a variável de estado e sua função atualizadora

03

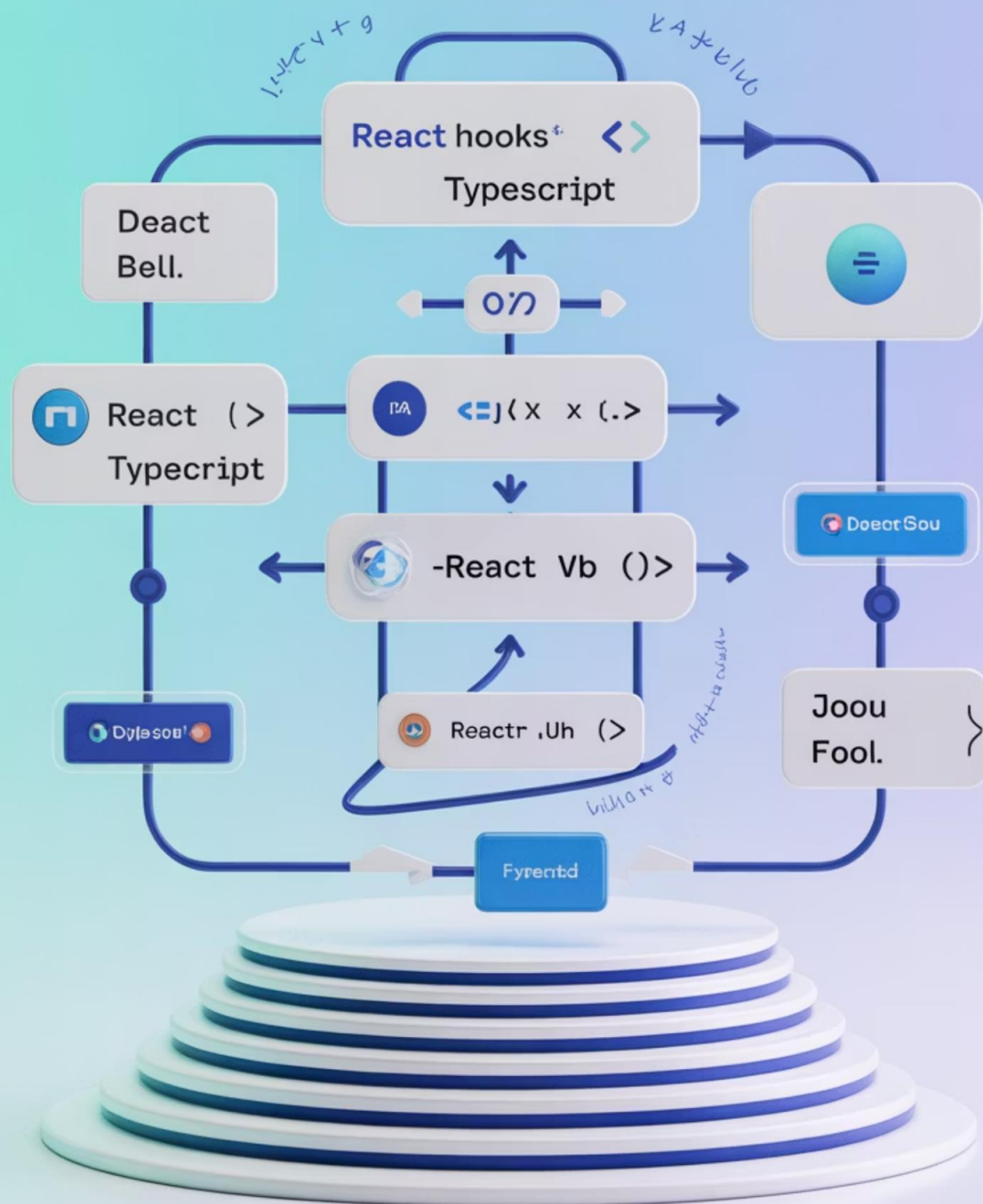
Valor Inicial

- Definir o valor inicial do estado dentro do useState

04

Utilização no Componente

- Usar a variável e a função



Por que usar useState no React Native?



Simplicidade e Poder

- Componentes funcionais ganham a capacidade de manter e gerenciar dados dinâmicos sem a complexidade adicional de classes



Performance Otimizada

- Hooks são otimizados pelo React, proporcionando melhor performance e facilitando a reutilização de lógica entre componentes



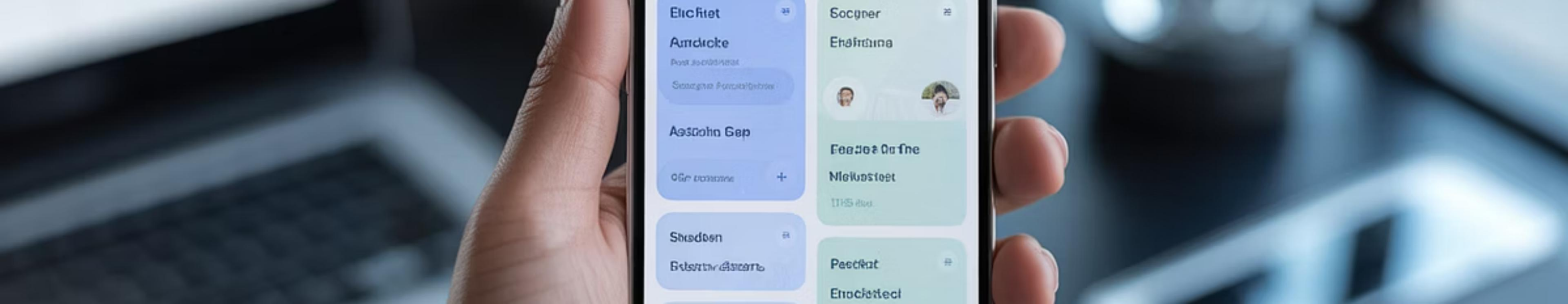
Código Mais Limpo

- Evita a verbosidade das classes e a confusão comum com `this.state` e `this.setState`, tornando o código mais legível



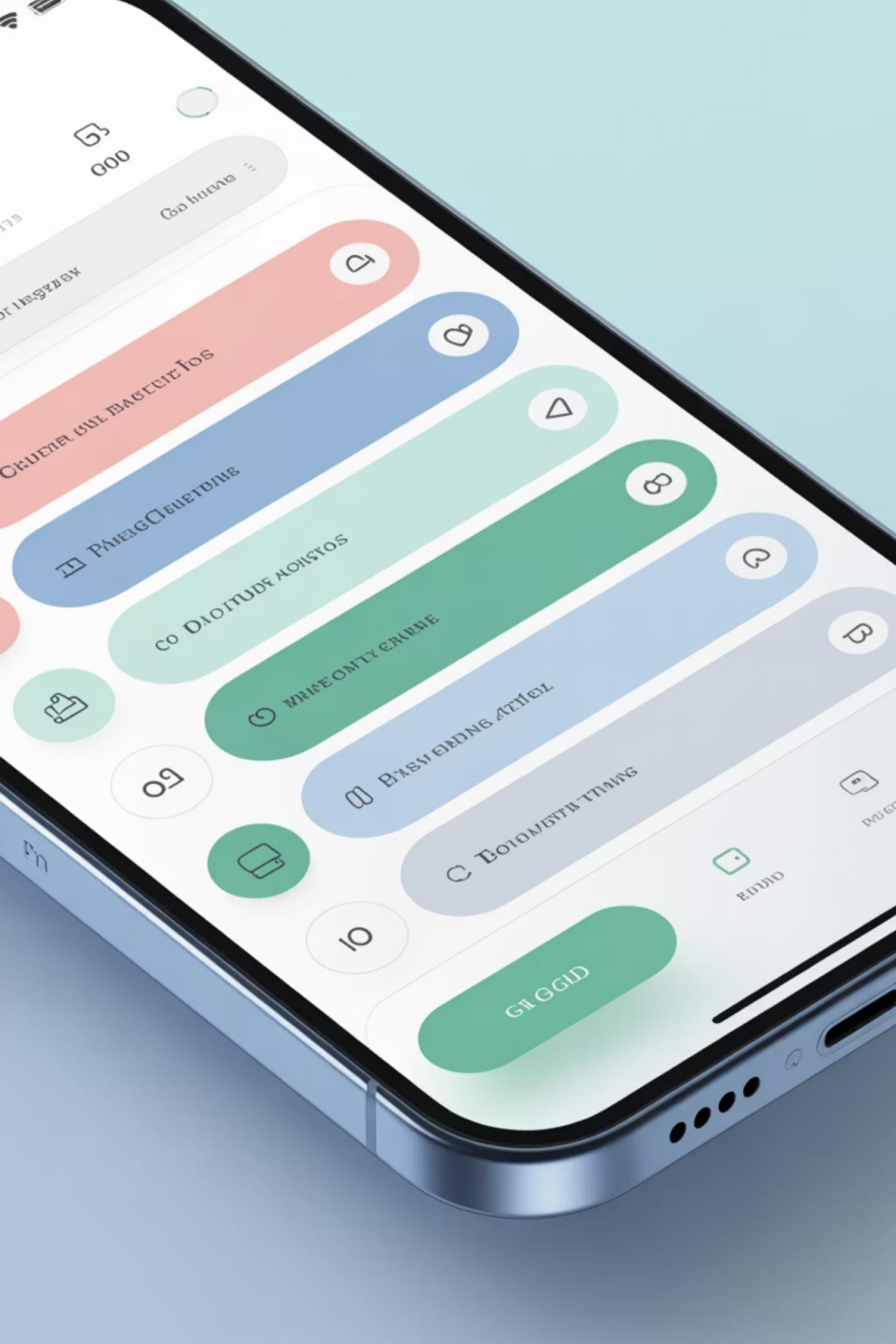
Composição Facilitada

- Permite combinar múltiplos estados e lógicas de forma modular, melhorando a organização do código



Exemplos Práticos no Expo Go

- Vamos explorar um exemplo prático que demonstram o poder do useState em aplicações React Native reais



Exemplo : Lista de Tarefas Interativa

- Um exemplo mais complexo que demonstra o uso de useState com arrays e objetos, permitindo adicionar e remover itens dinamicamente.

Criando o projeto

```
#Crie um novo projeto
```

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

```
npx create-expo-app prjListaDeTarefas --template blank
```

- Apague tudo e digite o código dos próximos 2 slides:

Código da Lista de Tarefas ...prjListaDeTarefas...Expo 54.0.0

```
JS App.js M X
prjListadeTarefas > JS App.js > ListaTarefas
You, há 32 minutos | 1 author (You)
1 import { StatusBar } from 'expo-status-bar';
2 import React, { useState } from 'react'; 7.9k (gzipped: 3.1k)
3 import { View, Text, TextInput, Button, FlatList, StyleSheet } from 'react-native';
4
5 import styles from './styles';
6
7 export default function ListaTarefas() { You, há 32 minutos • Uncommitted changes
8
9   const [tarefa, setTarefa] = useState('');
10  const [tarefas, setTarefas] = useState([]);
11  const adicionarTarefa = () => {
12    if (tarefa.trim()) {
13      setTarefas([...tarefas, { id: Date.now(), texto: tarefa }]);
14      setTarefa('');
15    }
16
17    return (
18      <View style={styles.container}>
19        <TextInput value={tarefa} onChangeText={setTarefa} placeholder="Digite uma tarefa" />
20        <Button title="Adicionar" onPress={adicionarTarefa}/>
21        <FlatList data={tarefas} renderItem={({item}) => <Text>{item.texto}</Text> } />
22      </View>
23    );
24  };
```

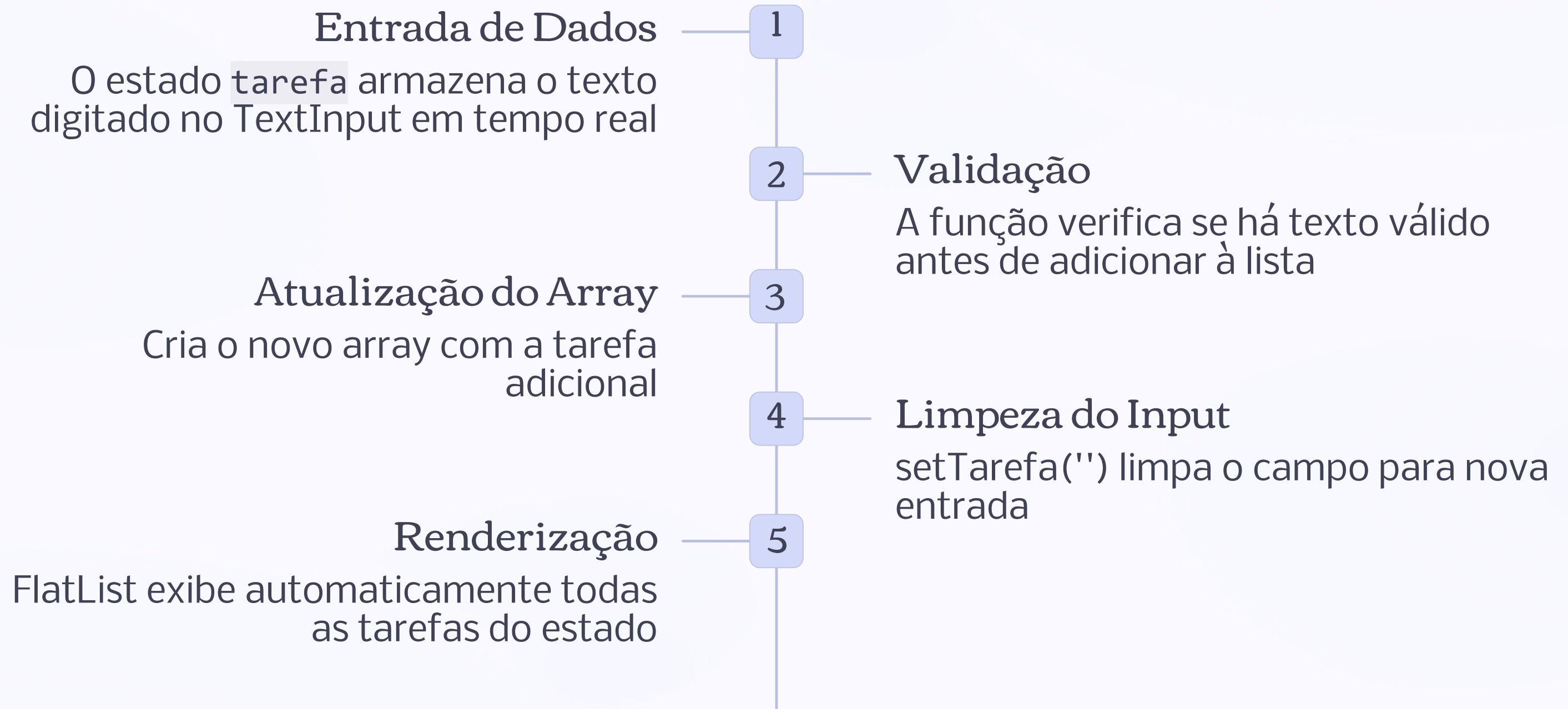
Código da Lista de Tarefas ...prjListaDeTarefas... Expo 54.0.0

JS styles.js U X

prjListadeTarefas > JS styles.js > ...

```
1  import {StyleSheet} from 'react-native';
2
3  const styles = StyleSheet.create({
4    container: {
5      flex: 1,
6      justifyContent: 'center',
7      alignItems: 'center',
8      padding: 120,
9    },
10 });
11
12 export default styles;
```

Fluxo da Lista de Tarefas



Conceitos-chave do Exemplo



Múltiplos Estados

- É possível usar vários `useState` no mesmo componente para gerenciar diferentes aspectos da aplicação de forma independente



Imutabilidade

- Sempre crie novos arrays ou objetos ao atualizar o estado



Reatividade

- Qualquer mudança no estado através da função `set` dispara automaticamente uma nova renderização do componente

O que é estado (state) em React Native?



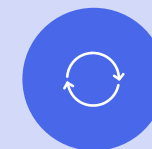
Armazenamento Dinâmico

Estado é uma forma de armazenar dados que podem mudar durante a vida do componente, como contadores, formulários e listas



Controle Local

Diferente de props (que vêm de fora), o estado é local e totalmente controlado pelo próprio componente



Re-renderização

Atualizar o estado provoca automaticamente a re-renderização do componente, mantendo tudo sincronizado



Atualização de Estado e Re-renderização

O ciclo de atualização do estado



Agendamento

- Quando `setValor` é chamado, React agenda uma nova renderização do componente



Re-execução

- O componente é re-executado completamente com o novo valor do estado atualizado



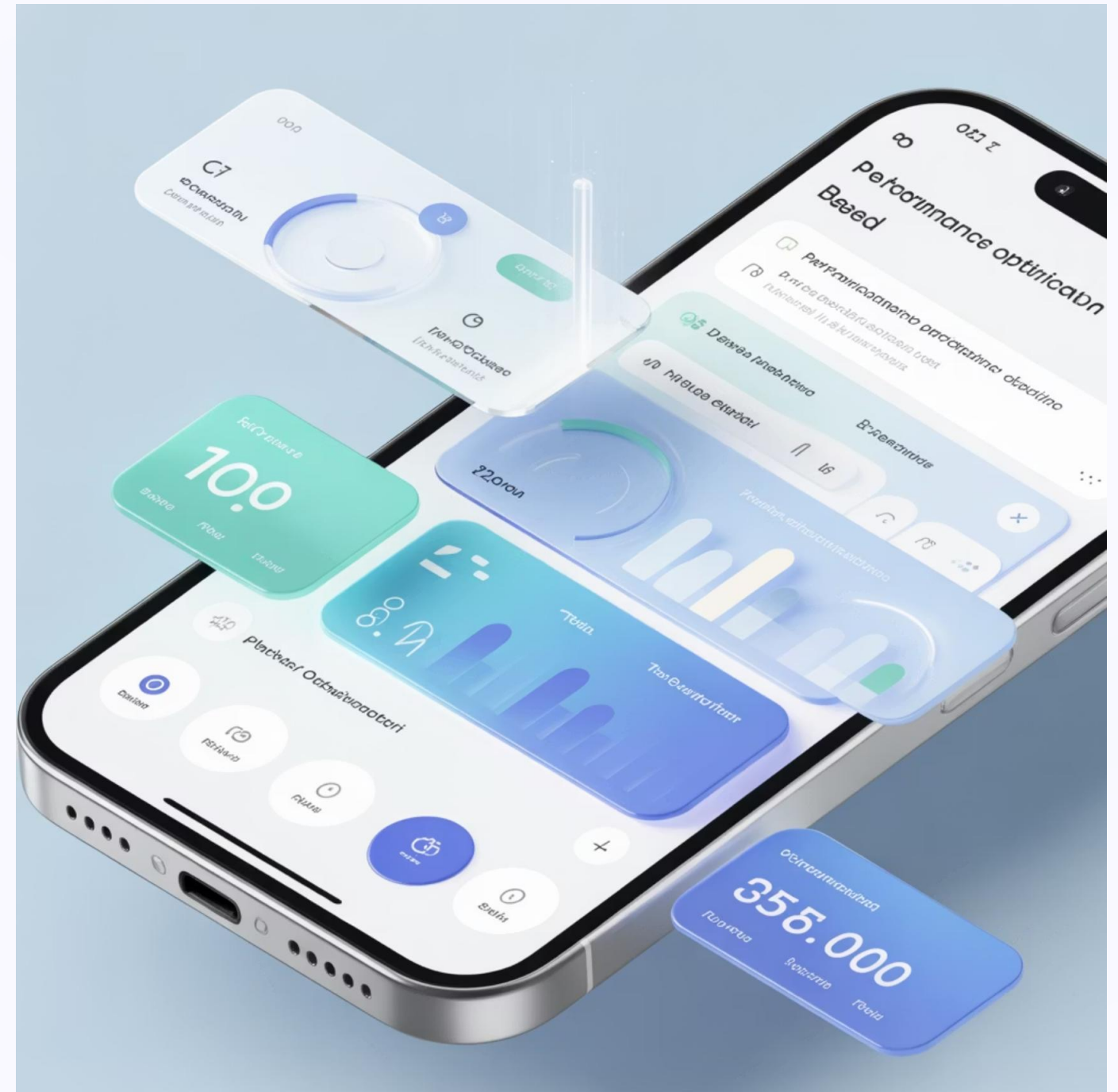
Otimização

- Apenas o componente que mudou e seus filhos diretos são atualizados na tela

Evitando re-renderizações desnecessárias

Como o React Otimiza

- React compara o estado antigo com o novo usando `Object.is`
- Se os valores forem idênticos, não há re-renderização
- Comparação rasa para objetos e arrays
- Essencial para performance em aplicações complexas



Exemplo prático: Lista dinâmica

Interface Interativa

- Código com useState para incrementar um número ao clicar no botão

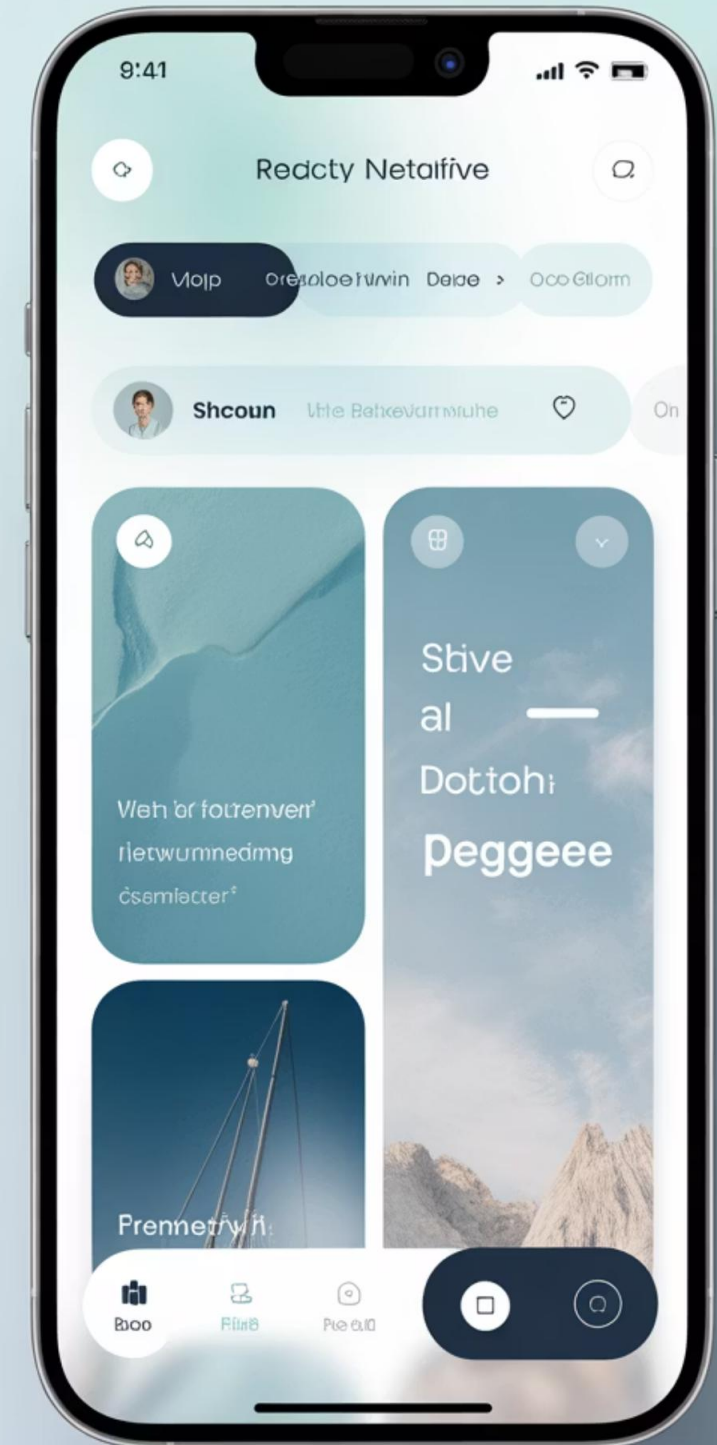
Atualização Instantânea

- Demonstração da atualização imediata da UI após cada clique

Testar no Expo Go

React Native: Listas com FlatList

Aprenda a criar listas eficientes e otimizadas no React Native usando o componente FlatList.



Por que usar FlatList?



Otimização Automática

Carrega apenas os itens visíveis na tela, economizando memória e processamento.



Performance Superior

Evita problemas comuns de performance com ScrollView e map().



Recursos Nativos

Suporta pull-to-refresh, múltiplas colunas e cabeçalhos prontos para uso.

FlatList vs ScrollView e map0

ScrollView + map0

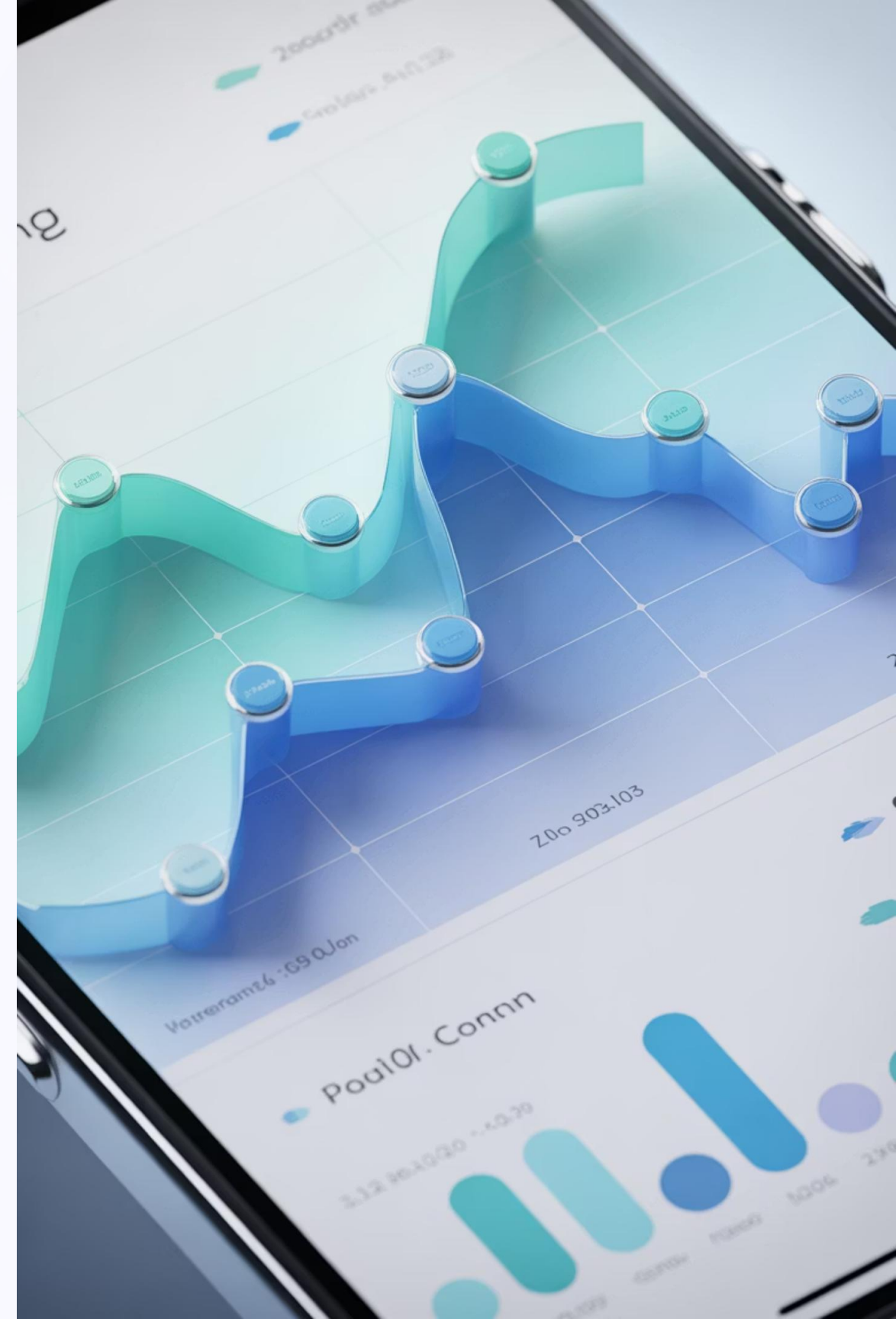
- ✗ Carrega todos os itens de uma vez
- ✗ Alto consumo de memória
- ✗ Lento com listas grandes

map0 simples

- ✗ Não cria lista rolável
- ✗ Sem otimização
- ✗ Requer ScrollView manual

FlatList

- ✓ Virtualização inteligente
- ✓ Renderiza só o necessário
- ✓ Performance otimizada



Propriedades Essenciais

Três propriedades fundamentais que você precisa dominar para usar o FlatList com eficiência.



Propriedade: data

A propriedade `data` recebe um array de objetos que será renderizado na lista.

Características:

- Aceita qualquer array de JavaScript
- Pode conter objetos, strings ou números
- Atualiza automaticamente quando o array muda

```
const dados = [ { id: '1', nome: 'Ana' }, { id: '2', nome: 'Bruno' }, { id: '3', nome: 'Carlos' } ];  
<FlatList data={dados} />
```

Propriedade: renderItem

A função `renderItem` define como cada item do array será exibido na tela. Ela recebe um objeto com o item e seu índice.

```
renderItem=(({ item, index }) => (  
  <View style={styles.item}>  
    <Text>{item.nome}</Text>  
  </View>  
))
```

Parâmetro: item

O objeto atual do array sendo renderizado.

Parâmetro: index

A posição do item no array (0, 1, 2...).



Propriedade: `keyExtractor`

A função `keyExtractor` retorna uma chave única para cada item, essencial para a performance do React.

Por que é importante?

- Ajuda o React a identificar mudanças
- Melhora a performance de atualizações
- Evita bugs de renderização

```
keyExtractor={(item) => item.id}  
// ou com índice  
keyExtractor={  
    (item, index) => item.id || index.toString()  
}
```

Criando o projeto

```
#Crie um novo projeto
```

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

```
npx create-expo-app prjListaDinamica --template blank
```

- Apague tudo e digite o código dos próximos 2 slides:

Exemplo prático : Lista dinâmicaprjListaDinamica

```
JS App.js M X
prjListaDinamica > JS App.js > ListaDinamica > renderItem
You, há 6 minutos | 1 author (You)
1 import { StatusBar } from 'expo-status-bar';
2 import React, { useState } from 'react'; 7.9k (gzipped: 3.1k)
3 import {
4   View,
5   Text,
6   TextInput,
7   TouchableOpacity,
8   FlatList,
9   StyleSheet
10 } from 'react-native';
11
12 import styles from './styles';
13
14 Explain Code | Generate Tests | Generate Docstrings | Ask Sourcery
15 export default function ListaDinamica() {
16   const [items, setItems] = useState([]);
17   const [texto, setTexto] = useState('');
18
19   const adicionarItem = () => {
20     if (texto.trim()) {
21       setItems([...items, { id: Date.now(), text: texto }]);
22       setTexto('');
23     }
24   };
25
26   const removerItem = (id) => {
27     setItems(items.filter(item => item.id !== id));
28   };
29 }
```

```
30 const renderItem = ({ item }) => ([
31   <View style={styles.itemContainer}>
32     <Text style={styles.item}>{item.text}</Text>
33     You, há 7 minutos • Uncommitted changes
34     <TouchableOpacity
35       style={styles.botaoRemover}
36       onPress={() => removerItem(item.id)}
37     >
38     <Text style={styles.textoBotaoRemover}>Remover</Text>
39   </TouchableOpacity>
40 </View>
41 ]);
42
43 return (
44   <View style={styles.container}>
45     <TextInput
46       style={styles.input}
47       value={texto}
48       onChangeText={setTexto}
49       placeholder="Digite um item"
50     />
51     <TouchableOpacity style={styles.botao} onPress={adicionarItem}>
52       <Text style={styles.textoBotao}>Adicionar</Text>
53     </TouchableOpacity>
54     <FlatList
55       data={items}
56       keyExtractor={item => item.id.toString()}
57       renderItem={renderItem}
58     />
59   </View>
60 );
61 }
62
63
64
65 }
```

Exemplo prático : Lista dinâmica

```
JS styles.js U X
prjListaDinamica > JS styles.js > ...
1 const styles = StyleSheet.create({
2   container: {
3     flex: 1,
4     marginTop: 40,
5     padding: 20,
6     backgroundColor: '#FFF'
7   },
8   input: {
9     borderWidth: 1,
10    borderColor: '#999',
11    borderRadius: 5,
12    padding: 10,
13    marginBottom: 10
14  },
15  botao: {
16    backgroundColor: '#007bff',
17    padding: 12,
18    borderRadius: 5,
19    marginBottom: 20,
20    alignItems: 'center'
21  },
22  textoBotao: {
23    color: '#FFF',
24    fontSize: 16,
25    fontWeight: 'bold'
26  },
```

```
27   itemContainer: {
28     flexDirection: 'row',
29     justifyContent: 'space-between',
30     alignItems: 'center',
31     backgroundColor: '#EEE',
32     padding: 12,
33     borderRadius: 5,
34     marginBottom: 10
35   },
36   item: {
37     fontSize: 16
38   },
39   botaoRemover: {
40     backgroundColor: '#d9534f',
41     paddingVertical: 6,
42     paddingHorizontal: 12,
43     borderRadius: 5
44   },
45   textoBotaoRemover: {
46     color: '#FFF',
47     fontWeight: 'bold'
48   }
49 });
50
51 export default styles;
```



Boas Práticas com useState em React Native

Dicas para usar useState corretamente

Nunca Modifique Diretamente

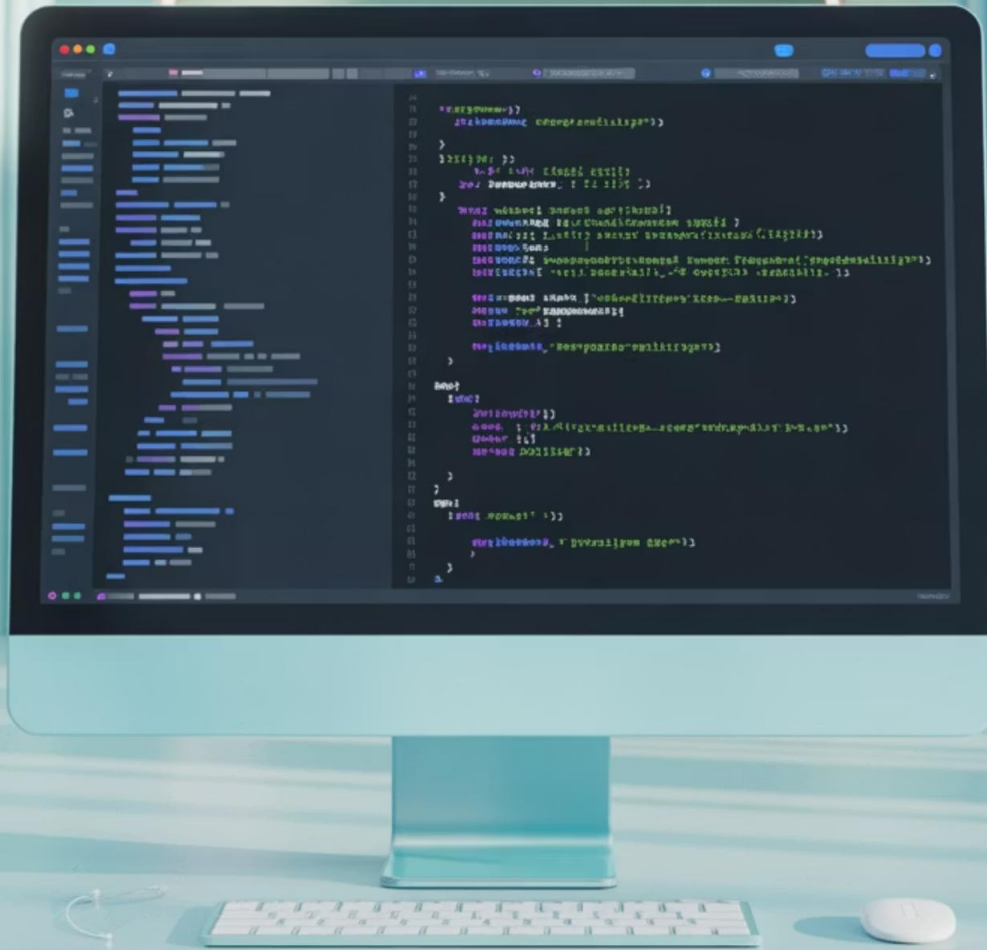
- Sempre use a função `setEstado`. Nunca faça `estado.push()` ou `estado = novoValor`

Divida Estados Complexos

- Para estados muito complexos, considere dividir em múltiplos `useState` menores

Evite Loops no Render

- Não chame `setEstado` dentro de loops ou condições durante a renderização - isso causa loops infinitos



Lidando com estados derivados e efeitos colaterais

Sincronização com useEffect

Use useEffect para sincronizar estado com APIs externas ou armazenamento local como AsyncStorage.

```
useEffect(() => {  
    AsyncStorage.setItem(  
        'contador',  
        count.toString()  
    );  
},  
[count]);
```

01

Estado Atualiza

- Usuário interage e muda o estado

02

useEffect Detecta

- Dependência dispara o efeito

03

Persiste Dados

- Salva no AsyncStorage



Explorando além do básico

Debugging e erros comuns

Estado não atualiza?

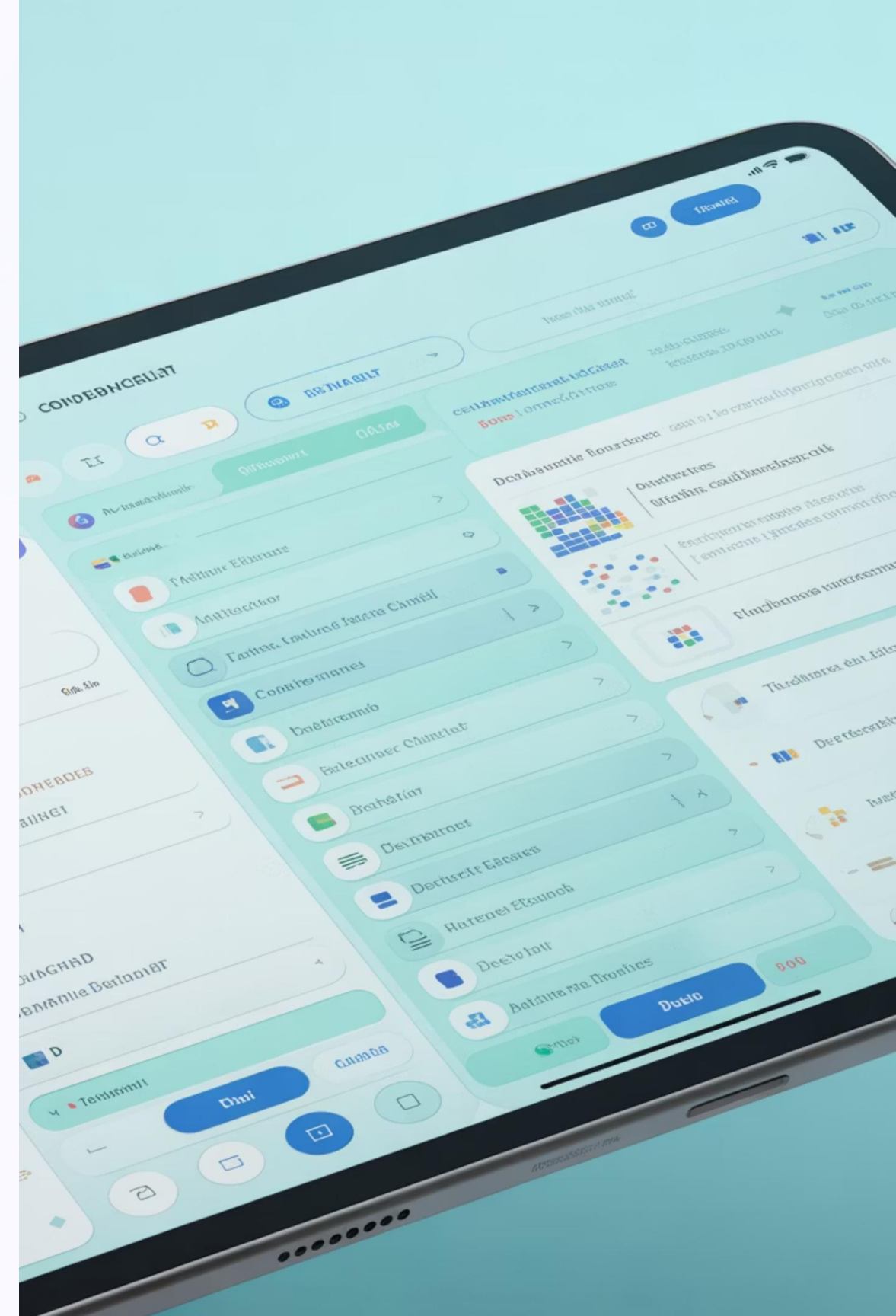
- Verifique se `setEstado` está sendo chamado
- Confirme que não está modificando diretamente
- Use `console.log` antes e depois

Valores antigos

- Cuidado com a captura de valores antigos
- Use função de atualização:
`setState(prev => prev + 1)`

Ferramentas de Debug

- React Native DevTools
- `console.log` estratégico
- Expo DevTools no navegador



Atualização baseada no estado anterior

Por que usar funções?

- Passar uma função para `setEstado` garante que você sempre trabalhe com o valor mais recente, especialmente importante em múltiplas chamadas rápidas.

📌 **Regra de Ouro:** Se a nova versão do estado depende da anterior, sempre use a forma funcional (arrow)

❌ Evite isso:

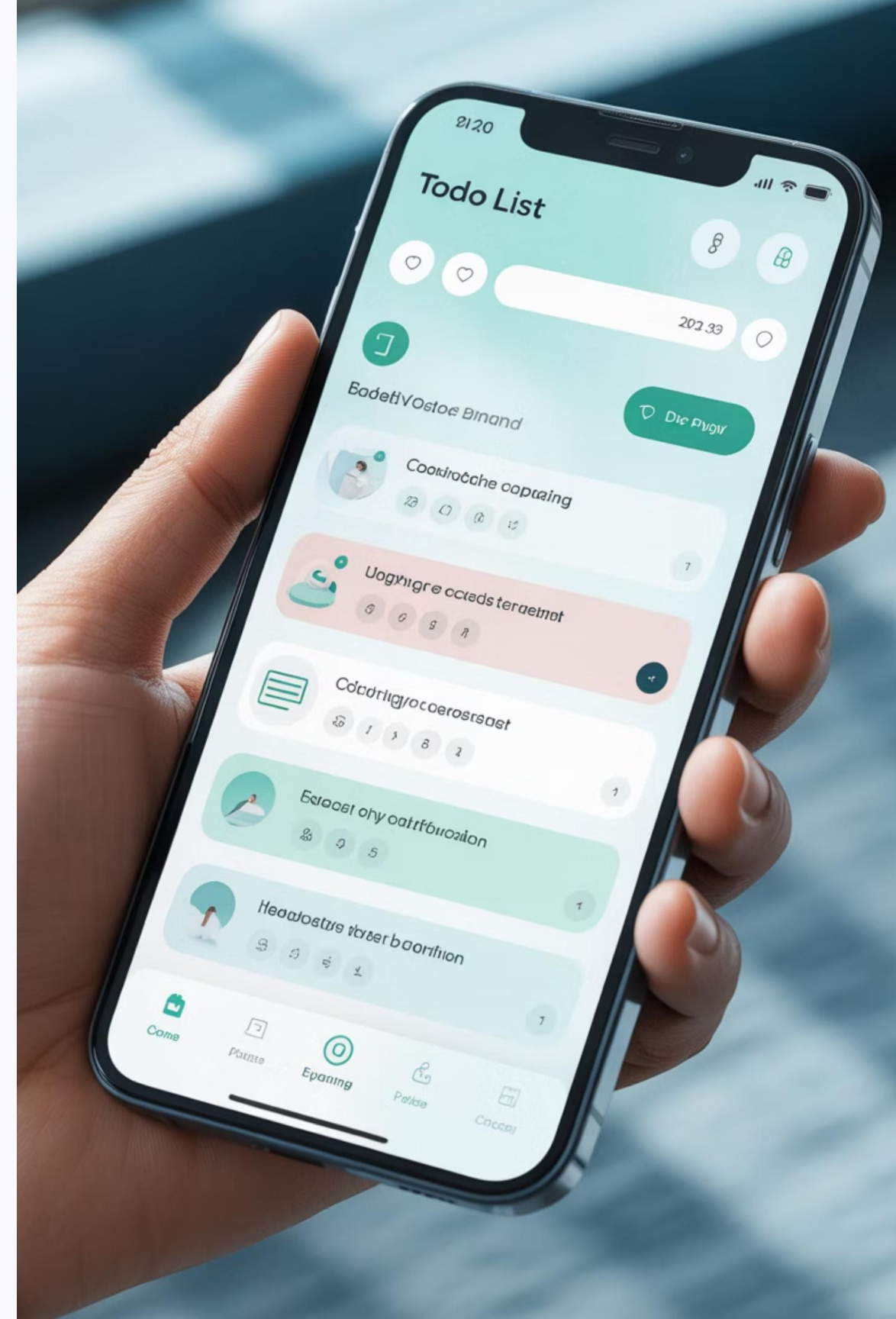
```
const [count, setCount] = useState(0);
setCount(count + 1);
setCount(count + 1); // Resultado: count + 1
```

✅ Prefira isso:

```
const [count, setCount] = useState(0);
setCount(prev => prev + 1);
setCount(prev => prev + 1); // Resultado: count + 2
```

Desafio Prático

Lista de Tarefas Dinâmica



Funcionalidades da Lista de Tarefas

1

Adicionar Tarefas

- Usuário pode criar novas tarefas dinamicamente

2

Marcar como Concluída

- Alterar status de tarefas com um toque

3

Remover Tarefas

- Excluir tarefas da lista permanentemente

- Tudo gerenciado com **useState** para controle de estado eficiente

Estrutura do Projeto no Expo

1

Criar App

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned  
expo create-expo-app prjListaDeTarefasDinamica --Template Blank
```

2

Componentes Principais

- App.js
- TodoList.js
- TodoItem.js

3

Testar no Expo Go

Visualização em tempo real no dispositivo

Estado Inicial das Tarefas ...prjListaDeTarefasDinamica

App.js

```
JS App.js M X
prjListaDeTarefasDinamica > JS App.js > ...
You, há 1 minuto | 1 author (You)
1 import { StatusBar } from 'expo-status-bar';      You, há 2 meses • Initial
2 import React, { useState, useEffect } from 'react'; 7.9k (gzipped: 3.1k)
3 import {
4   View,
5   TextInput,
6   Button,
7   FlatList,
8 } from 'react-native';
9
10 import styles from './styles';
11
12 import TodoItem from './TodoItem';
13
14 Explain Code | Generate Tests | Generate Docstrings | Ask Sourcery
15 export default function App() {
16   const [tarefas, setTarefas] = useState([]);
17   const [texto, setTexto] = useState('');
18
19   const adicionarTarefa = () => {
20     if (texto.trim()) {
21       setTarefas([
22         ...tarefas,
23         { id: Date.now(), texto, concluida: false }
24       ]);
25       setTexto('');
26     }
27   };
28 }
```

```
29 const toggleTarefa = (id) => {
30   setTarefas(
31     tarefas.map(t => {
32       if (t.id === id) {
33         return { ...t, concluida: !t.concluida };
34       }
35       return t;
36     })
37   );
38 };
39
40 const deletarTarefa = (id) => {
41   setTarefas(tarefas.filter(t => t.id !== id));
42 };
43
44 return (
45   <View style={styles.container}>
46     <TextInput
47       style={styles.input}
48       placeholder="Digite uma nova tarefa"
49       value={texto}
50       onChangeText={setTexto}
51     />
52     <Button title="Adicionar" onPress={adicionarTarefa} />
53     <FlatList
54       data={tarefas}
55       keyExtractor={(item) => item.id.toString()}
56       renderItem={({ item }) => (
57         <TodoItem
58           tarefa={item}
59           onToggle={() => toggleTarefa(item.id)}
60           onDelete={() => deletarTarefa(item.id)}
61         />
62       )
63     )
64   />
65 )
66 />
67 </View>
68 );
69
70 };
```

Estado Inicial das Tarefas

Styles.js

```
JS styles.js U X
prjListaDeTarefasDinamica > JS styles.js > [🔗] default
1  import { StyleSheet } from 'react-native';
2
3  const styles = StyleSheet.create({
4    container: {
5      marginTop: 80,
6      flex: 1,
7      padding: 20
8    },
9    input: {
10     borderWidth: 1,
11     padding: 8,
12     marginBottom: 10
13   }
14 });
15
16 export default styles;
```

Estado Inicial das Tarefas

TodoItem.js

```
TodoItem.js > ...
1  import React from 'react'; 7.9k (gzipped: 3.1k)
2  import {
3    View,
4    Text,
5    Button,
6    TouchableOpacity,
7    StyleSheet
8  } from 'react-native';
9
10 export default function TodoItem({ tarefa, onToggle, onDelete }) {
11   return (
12     <View style={styles.item}>
13       <TouchableOpacity
14         onPress={onToggle}
15         style={{ flex: 1 }}
16         accessibilityRole="button"
17         accessibilityLabel="Marcar tarefa como concluída"
18       >
19         <Text style={tarefa.concluida ? styles.concluida : styles.texto}>
20           {tarefa.texto}
21         </Text>
22       </TouchableOpacity>
23
24       <Button title="Excluir" onPress={onDelete} />
25     </View>
26   );
27 }
```

```
29  const styles = StyleSheet.create({
30    item: {
31      flexDirection: 'row',
32      alignItems: 'center',
33      paddingVertical: 10
34    },
35    texto: {
36      fontSize: 16
37    },
38    concluida: {
39      fontSize: 16,
40      textDecorationLine: 'line-through',
41      color: 'gray'
42    }
43  });
```

Diagrama da Aplicação

- O estado vive no App
- O TodoItem apenas exibe
- O filho não altera estado diretamente





Responsabilidade de cada arquivo

Arquivo	Responsabilidade
App.js	Estado, regras de negócio, persistência, renderização da lista
TodoItem.js	Exibição de uma tarefa e interação visual
AsyncStorage / localStorage	Persistência (infraestrutura)

Dicas para Estudo



Seja “CURIOSO”:

Procure revisar o que foi estudado.
Pesquise as referências bibliográficas.



Seja “ANTENADO”:

Leia a próxima aula.



Seja “COLABORATIVO”:

Traga assuntos relevantes para a sala de aula.
Participe da aula.
Proponha discussões relevantes sobre o conteúdo.



Prof. Me. Wilson Lourenço

